WPT Day 3 Morning

Fri 23 Oct 26 · A centered login page (HTML structure, margin auto, gradients, input borders, hover + transition), flexbox layout, and a first-principles introduction to JavaScript, type safety, and TypeScript

---

Table of Contents

1. Login Page: HTML Structure
2. Centering the Container: Margin Auto
3. CSS Styling: Gradients, Fonts and Border Radius
4. Input Fields: Borders, Outline and Spacing
5. Login Button: Customization, Hover and Transition
6. Aligning Elements: Divs, BR and Display Flex
7. Flex Properties: Alignment and Sizing
8. JavaScript Introduction
9. Type Safety, Data Types and TypeScript
10. Lab Assignment: Multi-Section Flex Page
11. Combined Worked Example: Complete Login Page
12. Quick Revision Checklist
13. Practice Exercises
14. Practice Answers

Login Page: HTML Structure

The problem this page solves

Day 1 and Day 2 gave you forms, selectors, the box model and colour. None of that yet produced a *finished* UI piece — a single, self-contained card that looks like a real product login screen. The morning's goal is concrete and visual:

> A **centered white div on a dark (gradient) background** containing a login form.

Everything that follows — `margin: auto`, bottom-only borders, `:hover`, `transition`, flex — exists because that one card must look and behave correctly. Build the HTML skeleton first; style is layered on top of structure that already makes sense.

Full element breakdown inside the container

The page has one main container. Inside it, in this exact order:

1. An `<h2>` whose text is `Login` — the card's title.
2. A `<form>` holding two inputs:

    - `type="text"` with `placeholder="Enter Username"`
    - `type="password"` with `placeholder="Enter Password"`
3. A `<button type="submit">` with the text `Login`.
4. A `<p>` with the text `Need account?` followed by an `<a>` whose visible text is `Sign Up`.

The lecture initially built the form and only later noticed the submit button was missing — then added it. That is not a trivia detail: a form without a submit control is incomplete for the user, even if every input field is perfect.

Anchor `href`: never leave it blank

The Sign Up link's `href` is set to `#`, **not left blank**.

Choice	What the browser / SEO tools infer
`href=""` (blank)	Broken / empty link — hurts SEO; signals an unfinished or accidental anchor
`href="#"`	Intentional placeholder — "this is a link, destination not defined yet"
Real URL later	Replace `#` when the Sign Up page exists

> **Exam:** Leaving `href` blank = broken link, hurts SEO. Setting `href="#"` tells the browser the link is **intentional but not yet defined**.

Background: An `<a>` without a usable `href` is still clickable chrome, but crawlers and accessibility tools treat empty destinations as defects. `#` is the conventional "top of page / stub" target while you wire the real route.

Differentiating the container: `id="login-container"`

A real page can hold many `<div>`s (navbar, footer, sidebars, cards). A bare tag selector `div { … }` would style *all* of them. The login card therefore gets:

```
<div id="login-container">
    …
</div>
```

Day 2's rule applies unchanged: **ID for the one element that must be uniquely addressed**. The CSS will target `#login-container`, not every division on the page.

External stylesheet: `login-style.css`

Styles live in a separate file named `login-style.css`, linked from `<head>` exactly as Day 2 taught:

```
<link rel="stylesheet" type="text/css" href="login-style.css">
```

Three attributes, all required for the link the lecture used:

- `rel="stylesheet"` — relationship: this resource is a stylesheet
- `type="text/css"` — content is CSS text
- `href="login-style.css"` — relative path, same folder as the HTML

> **Pitfall:** Creating `login-style.css` and forgetting the `<link>` leaves the page completely unstyled. The file is inert until linked.

Centering the Container: Margin Auto

Start from the visual problem

You have a 400×400 (later ~450–500px wide, height reduced as needed) white card. By default a block-level `div` stretches full width of its parent — or, once given a fixed width, sits hard against the left. The login card must sit in the **horizontal centre** of the viewport at every screen width.

Temporary red background: see the box before you dress it

Before final colours, the lecture painted the container **red** solely to *visualise its dimensions*. A transparent or white box on a white page is invisible while you tune width, height and margin. Temporary loud colour is a debugging technique, not a design choice — remove it once the geometry is right.

Mechanism: `margin: 90px auto`

```
#login-container {
    width: 400px;
    height: 400px;
    margin: 90px auto;
}
```

Unpack the shorthand:

Value	Side	Meaning
`90px`	top (and, in 2-value shorthand, also bottom)	Fixed top offset — pushes the card down from the top of the page
`auto`	left and right	Browser calculates **equal** left and right margins

Background (first principles): For a block box with a **definite width** smaller than its containing block, if left and right margins are both `auto`, CSS distributes the leftover horizontal space equally — that *is* horizontal centering. If you hardcode `margin-left: 10px`, that 10px is correct on one monitor and wrong on every other width.

Why not hardcode left and right?

A fixed pixel value (e.g. `10px`) does **not** center across different screen sizes. `auto` tells the browser: *give equal margin on both sides regardless of screen width*. Resize the window — the card stays centered.

Critical distinction: two different "centers"

Technique	What it centers
`margin: … auto` (with a fixed width on the box)	The **div itself** within the page
`text-align: center`	The **content inside** the div (text, inline/inline-block children) — **not** the div on the page

These are two separate operations solving two separate problems. Centering the card on the page does **not** automatically center the heading and inputs inside it; centering the text inside does **not** move the card on the page.

> **Exam:** `margin: auto` centers the **div**; `text-align: center` centers **content inside** the div. Do not swap them in an answer.

> **Pitfall:** Applying `margin: auto` to an element that still has the default `width: 100%` (or `auto` filling the parent) leaves no leftover space to split — so nothing appears to move. The box needs an explicit width smaller than the parent for horizontal `auto` margins to show a centering effect.

CSS Styling: Gradients, Fonts and Border Radius

Body background: gradient, not flat colour

A flat `background-color` only supports solid fills. The lecture switched the body to a **gradient** using the `background` property (not `background-color`):

```
body {
    background: linear-gradient(#673AB7, #E91E63);
}
```

Mechanism:

- Property: `background` — the shorthand that can accept images and gradients
- Function: `linear-gradient(color1, color2)` — mixes two colours into a continuous blend
- Colours: chosen from the **Material UI colour palette**, **500 shades** (e.g. deep purple + red/pink)
- Process: open the Material UI colour picker, copy the hex, paste into CSS

> **Exam:** Gradients need `background` (or `background-image`), not `background-color`. `background-color` is solid only.

Login container must be explicitly white

```
#login-container {
    background: white;
}
```

Without this, the container **inherits** the body's background — so the "white card" would fill with the same purple–pink gradient and disappear into the page. Inheritance of background is the Day 2 `div` default biting again; override it deliberately.

Font via the universal selector

```
* {
    font-family: "Segoe UI Light";
}
```

`*` applies the typeface to **every** element. The lecture's choice is `"Segoe UI Light"` — a light weight of Segoe UI, common on Windows.

Background: As Day 2 stressed, `font-family` only works if that face exists on the visitor's system; otherwise the browser silently falls back. For a course demo on lab machines this is fine; production sites usually list fallbacks (`"Segoe UI Light", "Segoe UI", sans-serif`).

Border radius: button vs card

Element	Typical value	Effect
Login button	`border-radius: 15px`	Rounded corners on the control
Login container	`border-radius: 20px`	Card-like rounded appearance

Same property, two radii — the card is slightly rounder than the button so the hierarchy reads "soft panel, sharper control".

Input Fields: Borders, Outline and Spacing

Shared class: one rule for both fields

Both inputs share `class="form-input"` so **one** CSS rule styles username and password together — Day 2's class payoff applied to form controls.

```
<input type="text" class="form-input" placeholder="Enter Username">
<input type="password" class="form-input" placeholder="Enter Password">
```

Width is relative to the parent

```
.form-input {
    width: 65%;
}
```

65% means 65% of the **parent** (`#login-container`), not 65% of the full browser window. Percentage values in CSS always reference the containing block's width.

Padding and vertical spacing

```
.form-input {
    padding: 15px 10px;      /* top/bottom 15px, left/right 10px */
    margin-bottom: 25px;     /* gap between stacked fields */
}
```

- Padding increases **internal** space inside the fields (text sits away from the edges).
- `margin-bottom: 25px` creates vertical spacing **between** the fields.

Borders: remove all, then restore only the bottom

The lecture's technique for the modern "underline-only" input:

1. `border-width: 0` — removes all four borders at once.
2. Restore the bottom edge only, with the per-side properties:
   - `border-bottom-width`
   - `border-bottom-color`
   - `border-bottom-style: solid`

Result: only a bottom border is visible (e.g. initially `1px solid red`, later adjusted to the preferred colour).

CSS exposes the full per-side family: `border-left-color`, `border-top-width`, `border-bottom-style`, and so on. You can rebuild any one side without the other three.

```css
.form-input {
    border-width: 0;
    border-bottom-width: 1px;
    border-bottom-style: solid;
    border-bottom-color: #E91E63;
}
```

Outline vs border: the focus box

Clicking an input draws a browser **focus ring**. That ring is the **outline**, not the border.

Feature	Border	Outline
Part of the box model	Yes (takes space in layout calculations with `border-box` etc.)	No — drawn outside, does not affect layout size
Typical use here	The permanent underline under the field	Default focus highlight on click/focus
Removal	Restyle or zero the border properties	`outline: none`

```css
.form-input {
    outline: none;
}
```

> **Exam:** The box on click is the **outline**; the styled line underneath is the **border**. `outline: none` removes the default focus ring.

> **Pitfall:** Removing outline improves the visual match to a designed UI, but removes a default accessibility cue. In production you normally replace it with a custom `:focus` style rather than deleting focus indication entirely. The lecture removes it for the demo look.

Login Button: Customization, Hover and Transition

Targeting and base styles

The button gets `id="login-button"` for unique targeting:

```
#login-button {
    width: 30%;
    padding: 12px 10px;            /* top/bottom 12, left/right 10 */
    border: 0;                     /* remove default black button border */
    background: #F44336;           /* Material UI red, 500 shade */
    color: white;                  /* contrast on red */
    margin-top: 10px;              /* spacing above the button */
    border-radius: 15px;
}
```

Hover: the `:hover` pseudo-class

A **pseudo-class** attaches a state to a selector. `:hover` means "while the pointer is over this element":

```
#login-button:hover {
    background: white;
    color: red;
    border: 1px solid red;
}
```

Effect: colours **invert** when the mouse moves over the button — red fill / white text becomes white fill / red text with a red outline.

Without any transition, that change is **instant and abrupt**.

Smooth change: `transition`

```
#login-button {
    transition: background 300ms;
}
```

Mechanism:

- Property: `transition`
- Declares **which** CSS property to animate (`background`) and **how long** the change takes
- Time unit: **milliseconds** — `1000ms = 1 second` , `500ms = 0.5 s` , `300ms = 0.3 s`
- Higher duration = slower, more visible; **300ms** is smooth but snappy

> **Exam:** Use the `background` shorthand (not only `background-color`) for compatibility with `transition` as the lecture specified — `transition: background 300ms` .

Background: `transition` does not invent new colours; it interpolates from the pre-hover computed value to the `:hover` value over the given duration. No `:hover` rule → nothing to transition to.

Aligning Elements: Divs, BR and Display Flex

Problem 1: inputs and button on the same line

Inside a form, inputs and buttons can sit on the same line depending on default display and whitespace. The lecture needed each control on its **own** line.

Rejected solution: `<br>`

`<br>` is only appropriate **inside paragraph or span elements** to break text lines. It is **not recommended** for aligning block-level HTML elements onto new lines. Using `<br>` as a layout tool fights the document structure instead of expressing it.

Correct solution: wrap each control in its own `div`

`div` s are **block-level** by default — each tends to render on a new line. Wrap each input and the button in a separate `div` → each appears on its own line:

```
<form>
    <div>
        <input type="text" class="form-input" placeholder="Enter Username">
```

```
    </div>
    <div>
        <input type="password" class="form-input" placeholder="Enter Password">
    </div>
    <div>
        <button type="submit" id="login-button">Login</button>
    </div>
</form>
```

Structure expresses "these are separate rows"; CSS does not need `<br>` hacks.

Problem 2: two (or more) divs side by side

The opposite layout problem: several `div`s must share **one row**. By default `div`s stack vertically. `display: flex` on a **parent** overrides that.

Key rule: flex on the parent, not the children

```
WRONG mental model                  CORRECT model
──────────────────                  ─────────────
child { display: flex; }            parent { display: flex; }
(does not lay siblings in a row)    children automatically share a row
```

- Create a parent `div` (e.g. `id="container"`) wrapping the child `div`s
- Apply `display: flex` to `#container`
- All direct child `div`s now appear in the **same row**

`display` property overview (as taught)

Value	Effect
`display: none`	Hides the element entirely (removed from layout)
`display: flex`	Makes the element a flex **container**; children become flex items in a row (default main axis)

> **Exam:** Flex must be applied to the **parent**, not the children themselves. Applying `display: flex` to individual sibling divs without a flex parent does **not** place those siblings side by side.

Flex Properties: Alignment and Sizing

`gap` : uniform spacing between flex items

```
#container {
    display: flex;
    gap: 20px;
}
```

`gap: 20px` adds uniform spacing between all flex items. It is cleaner than sprinkling margins on every child when the spacing should be identical.

`justify-content` : horizontal alignment (main axis)

Controls how flex items are distributed **along the main axis** (horizontal in the default row direction):

Value	Effect
`center`	All items centered horizontally
`flex-start`	Items aligned to the left (**default**)
`flex-end`	Items aligned to the right

`align-items` : vertical alignment (cross axis)

```
#container {
    display: flex;
    height: 500px;          /* needed so vertical centering is visible */
    align-items: center;
}
```

- `align-items: center` vertically centers items in the container
- Requires a **fixed height** on the container to have a visible effect — otherwise the container's height collapses to the items and there is no extra vertical space to center within
- Lecture tip: give the container a background colour while experimenting so you can **see** the 500px space

`flex` on children: distributing width

The `flex` property on children controls how **available width** is shared:

Setup	Maths	Result
All children `flex: 1`	Equal parts	Each gets an equal share of the row
`box1 flex:1`, `box2 flex:2`, `box3 flex:1`	Total parts = 1+2+1 = **4**	box1 = **25%**, box2 = **50%**, box3 = **25%**

Again: percentages and flex shares reference the **parent container**, not the full body.

```
#box1 { flex: 1; }
#box2 { flex: 2; }
#box3 { flex: 1; }
```

Responsive behaviour without media queries

Flex layout **reflows automatically** as the browser window is resized. Content does not distort; proportions are maintained. The lecture calls this a **basic form of responsive design**; **media queries** are the more advanced approach (taught later).

> **Exam:** `justify-content` = main axis (horizontal in a row); `align-items` = cross axis (vertical). `flex: 2` vs `flex: 1` means double the share of leftover space, not "2 pixels".

JavaScript Introduction

The problem HTML and CSS cannot solve

- **HTML** displays content (structure and meaning).
- **CSS** styles that content (appearance).
- Neither can **run logic**.

Without a logic language in the browser, a page cannot:

- Validate a form before submit
- Perform calculations
- Drive animations from user input beyond what CSS pseudo-states cover
- React in **real time** to what the user does

Concrete lecture example: **enable or disable a submit button** based on whether terms and conditions are accepted. That is a decision (`if` accepted → enable; else → disable). Decisions are JavaScript's job.

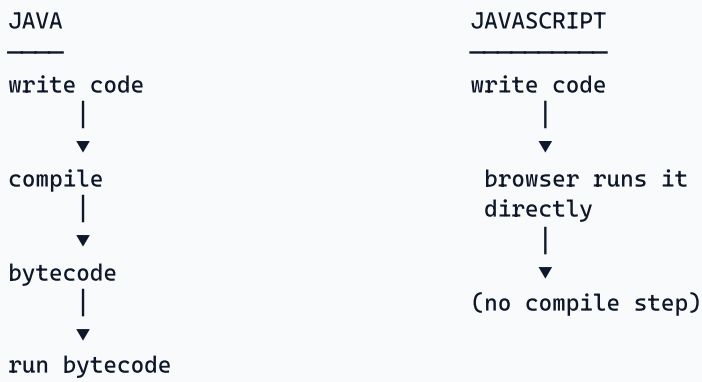
> **Exam:** Role of JavaScript = execute logic on the **client side** (in the browser). Without JS, a web page cannot interact with the user in real time in the sense the lecture means.

Name history (memorable, examinable)

Stage	Name / fact
Original name	**LiveScript** (late 1980s–90s era branding) — because it made pages "live"
Rename	Renamed to **JavaScript** because **Java** was dominant; the name attracted developer attention
Relation to Java	**Has nothing to do with Java** as a language
Later influence	Some Java-like syntax was intentionally introduced (`parseInt` , `class` , `extends`)
Accurate summary	Highly inspired by Java **syntactically**, but a **completely different language with a different purpose**

Execution model: no compiler required

Compare the two workflows the lecture contrasted:



- Every browser has a built-in **JavaScript engine** (interpreter): Chrome, Firefox, Brave, Opera, Edge all include one
- The engine **interprets and executes** JS at runtime inside the browser
- No JDK-style installation is needed for page scripts — the interpreter is **already bundled** with the browser

Background: Modern engines also JIT-compile hot code for speed; the lecture's teaching point is the **developer workflow** — you do not run a separate compile step before the browser can execute your `.js` , unlike Java's explicit compile-to-bytecode cycle.

Type Safety, Data Types and TypeScript

No type safety in JavaScript

In Java / C / C++:

```
int a = 5;      // a may only hold integers
a = "hello";    // compile-time error
```

In JavaScript, a variable can hold a **number today and a string tomorrow** with no error. The lecture's risk example: a variable meant to store "number of views" (numeric) could be accidentally reassigned a string — introducing **silent bugs**.

Data types exist — but they are dynamic

Predefined types include `number`, `string`, `boolean`.

- Types are assigned at **runtime** based on the current value, not locked by a developer declaration
- If `a = 5`, its type is `number`; if later `a = "hello"`, its type becomes `string`
- There is **no way to lock** a variable to a specific type in plain JavaScript

TypeScript: the enterprise answer

Idea	Detail
Definition	**TypeScript = JavaScript + static typing + additional OOP features**
What it adds	Declare and **enforce** data types at development time
Name	"Type" + "Script" — the name reflects the addition
Real workflow	Write in TypeScript → **compile TS to JS** → deploy the **JS** build
Learning path	Learn JavaScript first; TS is not a new language from scratch
Overlap	If you know JS, you already know **~90%** of TS
What is new	How to **declare and annotate types** in TypeScript
Recommendation	**Always use TypeScript** for enterprise / production applications to avoid type-related bugs

> **Exam:** TypeScript is not a replacement runtime in the browser — browsers run **JavaScript**. TS is a development-time language that compiles down to JS. JS has dynamic types; TS adds static typing on top of JS.

> **Pitfall:** Saying "JavaScript has no data types" is wrong. It has types; it lacks **static** (locked-at-declaration) type safety.

Lab Assignment: Multi-Section Flex Page

The lecture closed with a design-minded lab (not logistics). Requirements, in order:

Mindset

Build a full multi-section webpage with a **consistent, thoughtful colour theme**. Think like a UI designer: choose background colours, font colours and font families intentionally — **avoid random colour choices**.

Section 1 — Navbar (full browser width)

- Logo / brand image on the **left**
- Navigation links on the **right**: Home, About Us, Contact Us, Services, Gallery
- Use `display: flex` with `justify-content` to place logo left and links right

Section 2 — Intro

- Centered heading
- 3–4 line paragraph description
- One button below the paragraph

Section 3 — Two-column row (flex)

- **Smaller** column: one email input + subscribe button
- **Larger** column: a paragraph or description text
  (Hint from the flex lesson: unequal `flex` values, e.g. `flex: 1` and `flex: 2`.)

Section 4 — Three equal-width sections

- `flex: 1` on each
- Each section: one image centered, one paragraph below the image

This assignment is the practical synthesis of the morning: flex parent, `justify-content`, `flex` shares, and deliberate Material-style colour choices.

Combined Worked Example: Complete Login Page

Two files in the same folder. Open `login.html` in a browser. This ties together structure, margin centering, gradient body, underline inputs, hover transition, and div wrappers for vertical stacking. A small flex demo row under the card replays `justify-content` and `flex` shares.

File 1: `login-style.css`

```css
/* Universal font — every element inherits Segoe UI Light when available. */
* {
    font-family: "Segoe UI Light", "Segoe UI", sans-serif;
    box-sizing: border-box;
}

/*
 * Gradient on body: use background (not background-color).
 * Colours: Material UI 500 — Deep Purple 500 + Pink 500.
 */
body {
    margin: 0;
    min-height: 100vh;
    background: linear-gradient(#673AB7, #E91E63);
}

/*
 * The white card. Fixed width so margin auto can center it.
 * Temporary tip from lecture: use background: red while measuring, then white.
 */
#login-container {
    width: 450px;
    margin: 90px auto;
    padding: 40px 30px 30px;
    background: white;              /* without this, inherits the gradient */
    border-radius: 20px;            /* card-like corners */
    text-align: center;             /* centers content INSIDE the card */
}

#login-container h2 {
    margin-top: 0;
    margin-bottom: 30px;
    color: #333;
}
```

```css
/* Shared class for both inputs — one rule, two fields. */
.form-input {
    width: 65%;                     /* 65% of #login-container, not the body */
    padding: 15px 10px;
    margin-bottom: 25px;
    border-width: 0;                /* strip all four borders */
    border-bottom-width: 1px;       /* restore bottom only */
    border-bottom-style: solid;
    border-bottom-color: #E91E63;
    outline: none;                  /* remove browser focus ring (outline ≠ border) */
    background: transparent;
    font-size: 16px;
}

#login-button {
    width: 30%;
    padding: 12px 10px;
    margin-top: 10px;
    border: 0;
    border-radius: 15px;
    background: #F44336;            /* Material UI Red 500 */
    color: white;
    font-size: 16px;
    cursor: pointer;
    transition: background 300ms;   /* smooth; 300ms = 0.3s */
}

/* Pseudo-class :hover — invert colours while pointer is over the button. */
#login-button:hover {
    background: white;
    color: red;
    border: 1px solid red;
}

#login-container p {
    margin-top: 25px;
    color: #555;
}

#login-container a {
    color: #673AB7;
    text-decoration: none;
}

/* --- Flex demo strip under the card (lab concepts in miniature) --- */
#flex-demo {
    display: flex;                  /* flex on PARENT */
    gap: 20px;
    width: 450px;
    margin: 20px auto 40px;
    justify-content: center;        /* main axis: horizontal center */
```

```css
}

#flex-demo .box {
    flex: 1;                        /* equal shares; try flex:2 on one box */
    padding: 16px;
    background: rgba(255, 255, 255, 0.9);
    border-radius: 12px;
    text-align: center;
}

#flex-demo .box-wide {
    flex: 2;                        /* double share: 1+2+1=4 → 25% / 50% / 25% */
}
```

File 2: `login.html`

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Login – WPT Day 3 Morning</title>
    <!-- External CSS: inert until linked. Same folder as this file. -->
    <link rel="stylesheet" type="text/css" href="login-style.css">
</head>
<body>

    <!-- Unique id so we style THIS card, not every div on the page. -->
    <div id="login-container">
        <h2>Login</h2>

        <form>
            <!-- Each control in its own div → block-level new line (no <br>). -->
            <div>
                <input
                    type="text"
                    class="form-input"
                    placeholder="Enter Username"
                    name="username">
            </div>
            <div>
                <input
                    type="password"
                    class="form-input"
                    placeholder="Enter Password"
                    name="password">
            </div>
            <div>
                <!-- type="submit" – lecture initially forgot this, then added it. -->
                <button type="submit" id="login-button">Login</button>
```

```
            </div>
        </form>

        <!-- href="#" = intentional stub; blank href = broken link / SEO harm. -->
        <p>Need account? <a href="#">Sign Up</a></p>
    </div>

    <!-- Parent is the flex container; children share one row. -->
    <div id="flex-demo">
        <div class="box">flex: 1</div>
        <div class="box box-wide">flex: 2 (50%)</div>
        <div class="box">flex: 1</div>
    </div>

</body>
</html>
```

Trace it end to end

1. **Link first.** The browser reads `<head>`, loads `login-style.css` via `<link>`. Delete that line and the gradient, card, underline inputs and hover all vanish — external CSS is inert until linked.
2. **The card is centered as a box, not as text.** `#login-container` has `width: 450px` and `margin: 90px auto`. Equal automatic left/right margins center the **div on the page**. Separately, `text-align: center` centers the **content inside** the card. Two problems, two properties — the lecture's distinction made visible.
3. **White on purpose.** `background: white` on the container stops inheritance of the body's `linear-gradient(#673AB7, #E91E63)`. Comment out that one line and the "card" dissolves into the purple–pink wash.
4. **Inputs share a class.** Both fields use `class="form-input"`: 65% of the **parent** width, padding 15px/10px, `margin-bottom: 25px`, borders stripped then bottom-only restored, `outline: none` so the click ring (outline) does not fight the underline (border).
5. **Button hover + transition.** Default state is Material red with white text. `#login-button:hover` inverts to white/red with a 1px red border. `transition: background 300ms` makes the fill change take 0.3 s instead of snapping.
6. **Structure instead of `<br>`.** Each input and the button sits in its own `div`, so block layout stacks them. The Sign Up anchor uses `href="#"` — intentional placeholder, not an empty broken link.
7. **Flex strip under the card.** `#flex-demo` has `display: flex` (on the **parent**), `gap: 20px`, `justify-content: center`. Children use `flex: 1`, `flex: 2`, `flex: 1` → total 4 parts → 25% / 50% / 25%, matching the lecture's box1/box2/box3 arithmetic. Resize the window: proportions hold — basic responsive behaviour without media queries.

Quick Revision Checklist

Topic	One-line takeaway
Login goal	Centered white div on dark/gradient background holding a login form
Structure	`h2` → form (text + password) → submit button → "Need account?" + Sign Up link
`href="#"`	Intentional stub link; blank `href` = broken link and hurts SEO
`#login-container`	ID differentiates this card from other `div` s on the page
External CSS	`login-style.css` linked with `rel` , `type` , `href` in `<head>`
Temporary red	Loud background only to visualise dimensions while tuning size/margin
`margin: 90px auto`	Fixed top; `auto` left/right = equal side margins = horizontal center
Why `auto`	Hardcoded left/right px fails across screen widths
Two centers	`margin: auto` → the div; `text-align: center` → content inside the div
Gradient	`background: linear-gradient(c1, c2)` — not `background-color`
Material UI	Pick 500-shade hex values from the palette and paste into CSS
Container white	Must set explicitly or it inherits the body gradient
Universal font	`* { font-family: "Segoe UI Light"; }`
Border radius	Button ~15px; container ~20px for card look
`.form-input`	Shared class; `width: 65%` of **parent**
Underline input	`border-width: 0` then restore `border-bottom-*` only
Outline vs border	Click ring = outline (`outline: none`); underline = border
`#login-button`	Width 30%, `border: 0` , Material red, white text, `margin-top: 10px`
`:hover`	Pseudo-class; invert colours on pointer-over
`transition`	e.g. `transition: background 300ms` — 1000ms = 1s
No ` ` for layout	` ` for text breaks in `p` / `span` , not for stacking controls
Stack controls	Wrap each in its own `div` (block → new line)

Topic	One-line takeaway
Flex rule	`display: flex` on the **parent**; children share a row
`display: none`	Hides element entirely
`gap`	Uniform spacing between flex items (cleaner than per-child margin)
`justify-content`	Main axis: `center` / `flex-start` (default) / `flex-end`
`align-items`	Cross axis; `center` needs a fixed container height to show
`flex: n`	Share of free space; 1+2+1 = 4 parts → 25% / 50% / 25%
Flex + resize	Reflows automatically — basic responsive design; media queries later
JS role	Client-side logic; HTML/CSS cannot run decisions
LiveScript	Original name; renamed JavaScript for marketing near Java's fame
JS ≠ Java	Different language and purpose; some borrowed syntax later
JS run model	No separate compile step; browser engine interprets at runtime
Dynamic types	Types exist (`number` , `string` , `boolean`) but can change anytime
TypeScript	JS + static typing + OOP extras; compile TS → deploy JS; ~90% is JS

Practice Exercises

Work these without looking at the answers. Most target confusions the lecture called out: blank `href` , `margin` vs `text-align` , `background-color` vs gradient, outline vs border, flex on the wrong element, and "JS has no types".

Concepts — login structure and centering

1. List the elements inside `#login-container` in lecture order, and state why the submit button matters even when both inputs already exist.
2. A student writes `<a href="">Sign Up</a>` . What goes wrong for SEO / tooling, and what should `href` be set to as a placeholder?
3. You set `margin-left: 200px; margin-right: 200px` on a 400px-wide card. Why is this a bad centering strategy compared to `margin: 90px auto` ?

4. True or false, with a reason: "`text-align: center` on the body will center the login `div` on the page."

Gradients, inputs and the button

5. Why does `background-color: linear-gradient(#673AB7, #E91E63);` fail, and what property should hold the gradient?
6. After painting `body` with a gradient, the login card also looks purple–pink. Name the inheritance issue and the one-line fix.
7. Describe the two-step border technique that leaves only an underline under `.form-input`.
8. A classmate says: "I removed the border, so why is there still a box when I click the field?" Correct the terminology and name the property that removes it.
9. Write the `#login-button:hover` effect the lecture used (colours and border), and explain what `transition: background 300ms` changes about that effect. How many milliseconds is half a second?

Flex and display

10. Why did the lecture reject `<br>` for putting the password field on a new line under the username field? What structure replaces it?
11. Spot the bug: three sibling `div`s each have `display: flex;` and the author expects them to sit in one horizontal row. What is wrong, and how do you fix it?
12. A flex container has three children with `flex: 1`, `flex: 2`, and `flex: 1`. What percentage of the row does each get? Show the arithmetic.
13. `align-items: center` seems to do nothing. Which other property must the container usually have for the centering to be visible, and why?
14. Match each to main axis or cross axis in a default row flex container: (a) `justify-content: flex-end` (b) `align-items: center`.

JavaScript and TypeScript

15. In one sentence each: what HTML does, what CSS does, and what JavaScript adds that the other two cannot.
16. Give the original name of JavaScript and the marketing reason it was renamed. Does that mean it is related to Java?
17. Contrast the Java compile workflow with the JavaScript browser workflow in three steps each.
18. A variable `views` starts as `100` and is later assigned `"many"`. Is this legal in JavaScript? What category of bug does the lecture say this invites? What tool/language is recommended for enterprise apps to prevent it, and what is the deployable output of that language?

Practice Answers

1. Inside the container, in order: `<h2>` ("Login"); `<form>` with text username and password inputs; `<button type="submit">` ("Login"); `<p>` ("Need account?") plus `<a>` ("Sign Up"). The submit button is the control that actually sends the form — the lecture initially forgot it and had to add it; inputs alone do not complete the login UI.

2. A blank `href` is treated as a **broken / empty link** and **hurts SEO**. Use `href="#"` as the intentional placeholder meaning "link is deliberate, destination not defined yet."

3. Fixed pixel side margins only look centered for **one** viewport width. `margin: … auto` lets the browser compute **equal** left and right margins for whatever width remains, so the card stays centered when the screen size changes.

4. False. `text-align: center` centers **content inside** a block (text / inline-level children). Centering the **div itself** on the page requires a definite width plus horizontal `margin: auto` (e.g. `margin: 90px auto`).

5. `background-color` only accepts **solid** colours; a gradient is not a solid. Put `linear-gradient( … )` on `background` (as the lecture did), e.g. `background: linear-gradient(#673AB7, #E91E63);`.

6. The container **inherits** the body's background if you never override it. Fix: `#login-container { background: white; }` (or equivalent solid fill).

7. First set `border-width: 0` to clear all four sides. Then restore only the bottom with `border-bottom-width`, `border-bottom-color`, and `border-bottom-style: solid`.

8. The click box is the **outline** (browser focus ring), not the border. Remove it with `outline: none`. The styled underline under the field is the **border**.

9. On hover: **background white**, **font colour red**, `1px solid red` **border** — colours invert. Without `transition` the change is instant; `transition: background 300ms` animates the background over **300ms**. Half a second = **500ms**.

10. `<br>` is for breaking **text** inside `p` / `span`, not for laying out block controls. Wrap each input/button in its **own** `div` so default block behaviour puts each on a new line.

11. Flex on each sibling does not arrange those siblings into one row — `display: flex` **must be on the parent**. Wrap them in a container and set `display: flex` on that parent; then the children become flex items in a row.

12. Total parts = 1 + 2 + 1 = **4**. First = 1/4 = **25%**, second = 2/4 = **50%**, third = 1/4 = **25%**.

13. Give the container a **fixed height** (lecture example: `height: 500px`). Vertical centering needs leftover space on the cross axis; if height hugs the content, there is nothing to center within. A temporary background colour helps you see that space.

14. (a) **Main axis** (horizontal in default `flex-direction: row`) — `justify-content`. (b) **Cross axis** (vertical) — `align-items`.

15. HTML displays/structures content. **CSS** styles appearance. **JavaScript** executes **logic** on the client (validation, calculations, real-time interaction) — which HTML and CSS cannot do.

16. Originally **LiveScript** (pages made "live"). Renamed to **JavaScript** because **Java was dominant** and the name attracted developers. It is **not** the same language as Java — different purpose; later some Java-like syntax was borrowed.

17. Java: write code → compile → generate bytecode → run bytecode. **JavaScript:** write code → browser runs it directly (no separate compile step) via the built-in JS engine/interpreter.

18. Yes, legal in JavaScript — types are dynamic. It invites **silent bugs** (e.g. a "number of views" variable quietly becoming a string). Use **TypeScript** for enterprise/production; workflow is write TS → **compile to JavaScript** → deploy the **JS** build. Knowing JS means you already know ~90% of TS; the new part is declaring/annotating types.